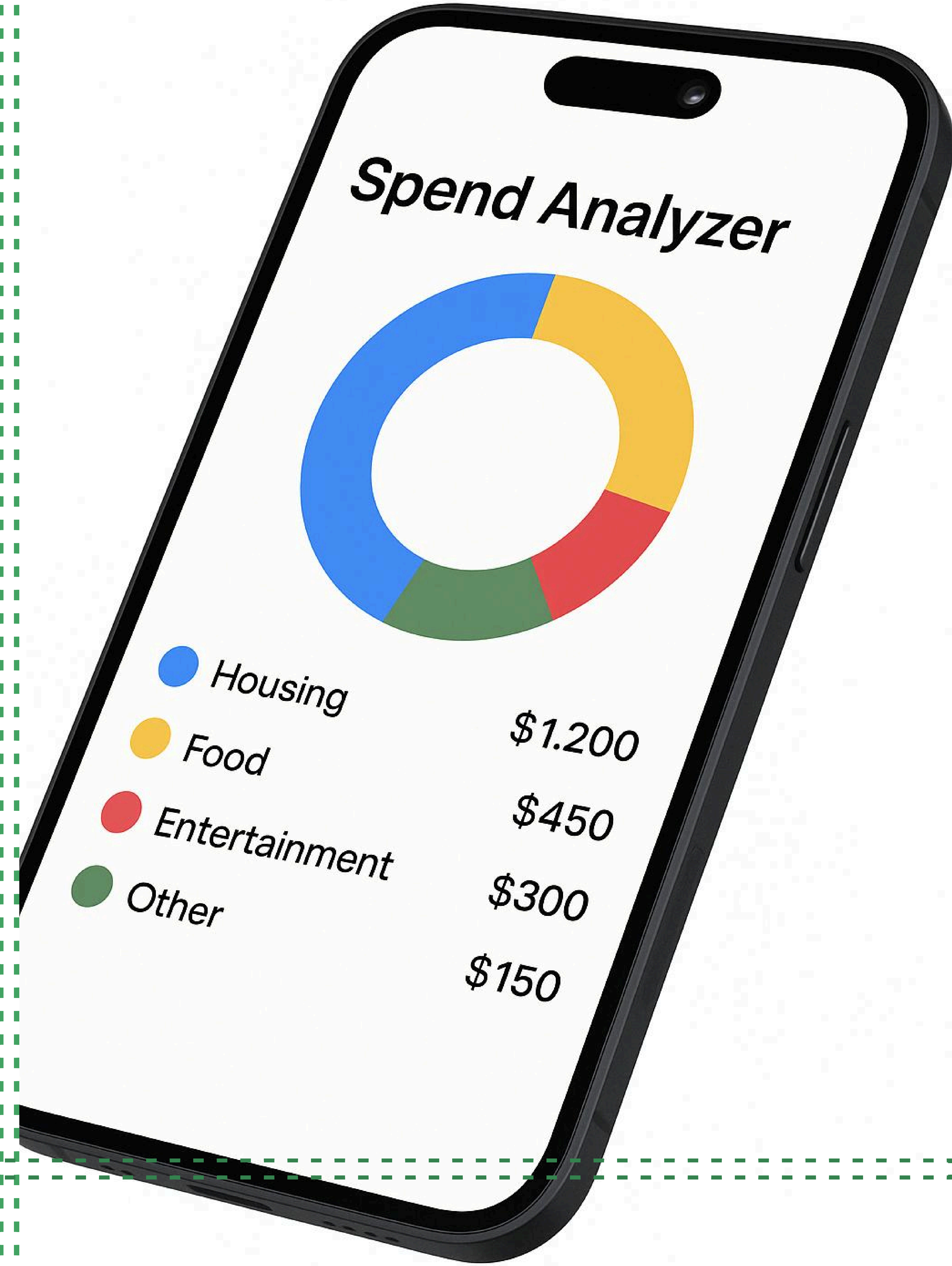
An AWS *Spend Analyzer* Project

recording of demo

Ananya Shah, Abigail Siy, Udita Shah



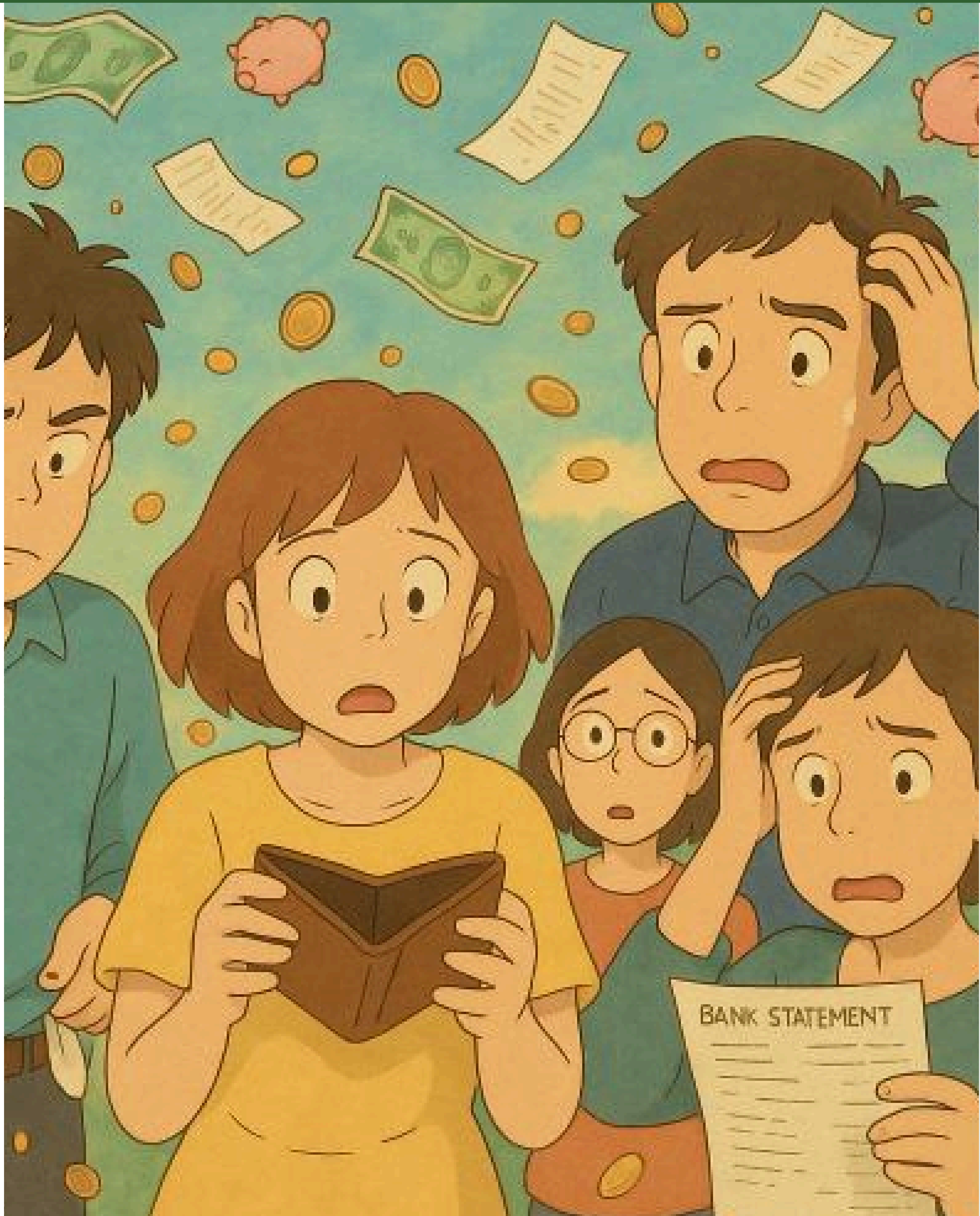
Spend Analyzer

Automated spending insights using AWS and Streamlit

Students and young professionals don't understand where their money goes

Budgeting tools can be expensive and require many API integrations

So, we created a tool which analyzes the spending habits of our target audience and provides them tips on saving based on benchmark spending data, all with a single bank statement upload.



Solution Overview

Upload → **Analyze** → **Suggest**

Upload

Users upload a bank statement (CSV/PDF)

Analyze

Transactions are cleaned (lambda), categorized, and stored (RDS)

Suggest

- Visual analytics show spending patterns
- Comparison to federal benchmarks
- Personalized savings suggestion

AWS services used & why

Service	Purpose	Why We Used It
S3	File ingestion + intermediate storage	Scalable object store to easily upload and process user bank statements
Lambda	Lightweight event-driven processing	Serverless execution for automatic data cleaning and database insertion
RDS (MySQL)	Persistent transaction storage	Structured relational storage for fast querying and analysis of cleaned data
EC2	Hosts the Streamlit app with Nginx reverse proxy	Flexible compute to run a customized Python dashboard accessible via the web
IAM	Secure communication between services	Granular access control to ensure least-privilege permissions across AWS stack

System Architecture (how they were used)

1. **EC2**

   Hosts Streamlit frontend which facilitates uploads & visualizes spend trends, categories, benchmark comparisons, and tips

2. **S3 (raw)**

   Stores uploaded bank statements

3. **Lambda 1**

   Parses + cleans transaction files, combines debit & credit to a single column, denoting transaction with '+' or '–'

4. **S3 (processed)**

   Triggers Lambda 2 with cleaned files

5. **Lambda 2**

   Categorizes and inserts transactions into RDS (MySQL)

6. **RDS**

   Stores normalized transaction data

Challenges & Fixes

The main challenges involved configuring secure and seamless communication between AWS services, ensuring event-driven triggers fired correctly, and maintaining compatibility with the Free Tier to keep the architecture lightweight and cost-effective.

- **Lambda didn't trigger** | Switched from copy_object() to upload_file()
- **RDS timeout error** | Fixed Security Group to allow EC2 access
- **Data normalization quirks** | Added flexible parsing logic
- **Keeping architecture within free tier** | Optimized instance type and Lambda memory

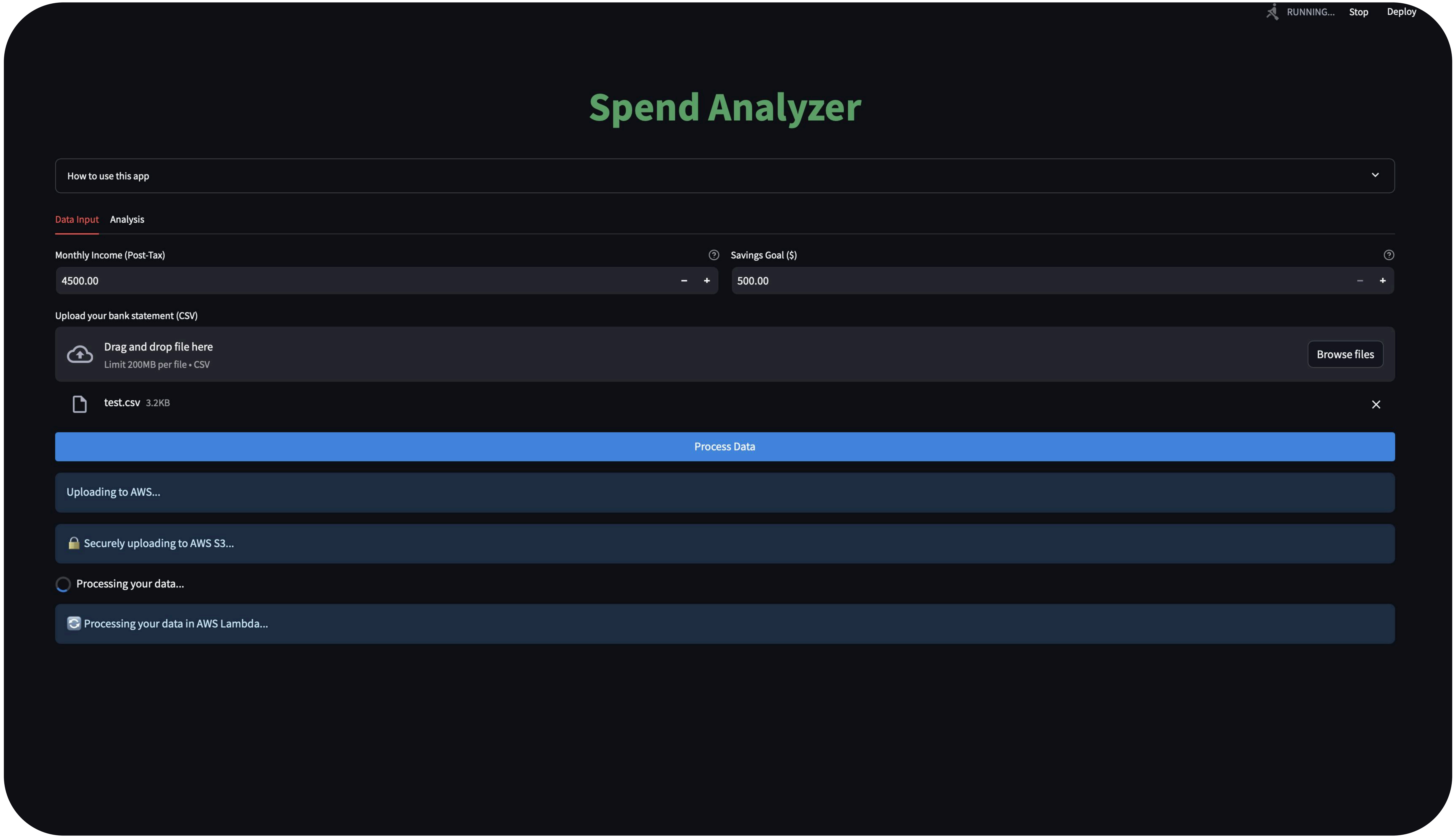
“

Extensive learning and research was required to get **Spend Analyzer** off the ground. Especially for:

- **Streamlit Deployment on EC2** – with Nginx reverse proxy and systemd service management
- **Troubleshooting Cloud Architectures** – especially debugging event propagation, timeouts, and network permissions
- **ML Pipelines in Streamlit**– Processing HG models within streamlit for categorizing data

Demo

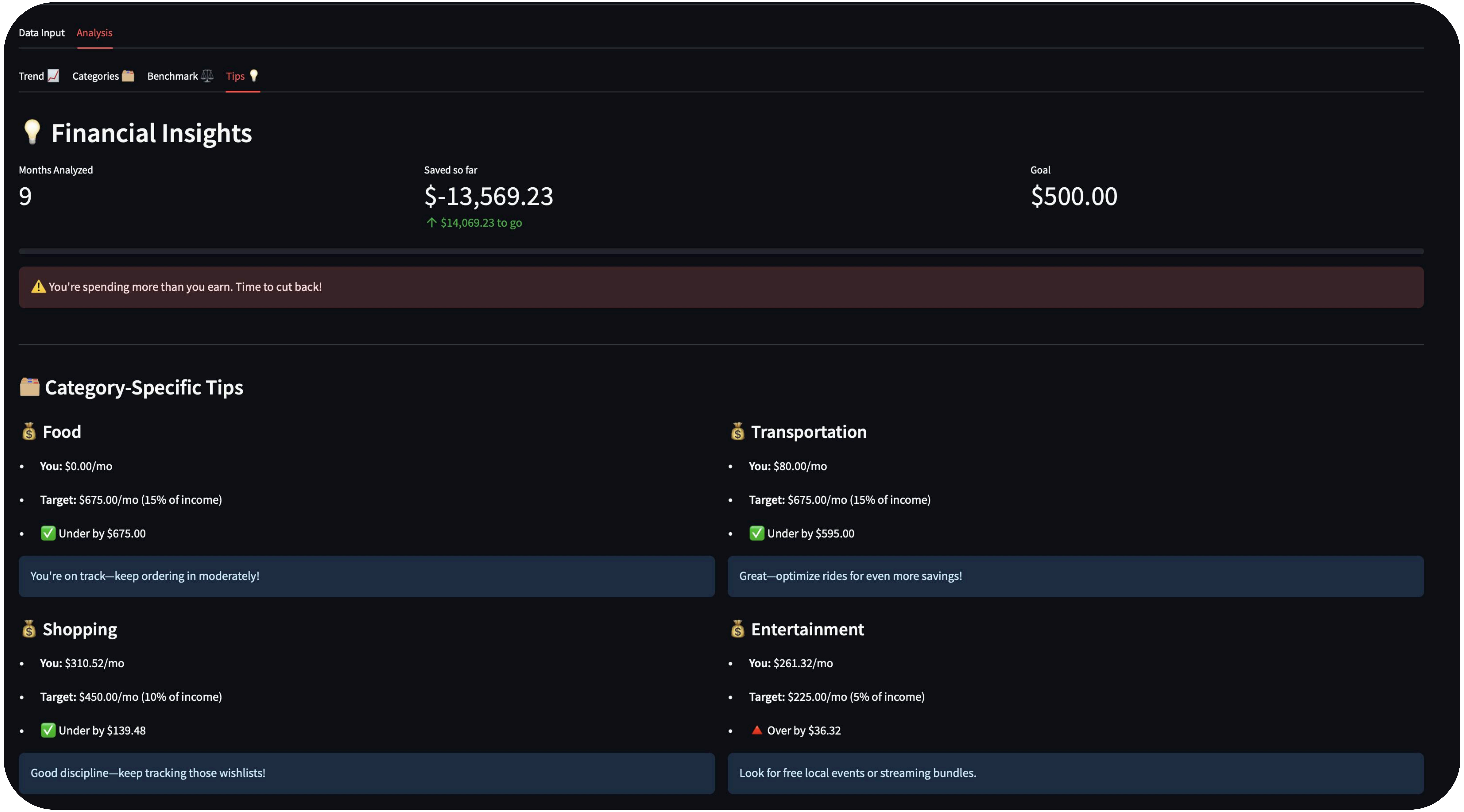
A peak into the screens of Spend Analyzer



uploading files

category	Total	Monthly Avg	% of Spending
Dining Out	$6,250.86	$694.54	12.6%
Entertainment	$2,351.87	$261.32	4.7%
Groceries	$12,500.84	$1,388.98	25.2%
Gym	$400.00	$44.44	0.8%
Health Insurance	$1,600.00	$177.78	3.2%
Internet	$400.00	$44.44	0.8%
Miscellaneous	$1,469.21	$163.25	3.0%
Personal Care	$1,641.79	$182.42	3.3%
Phone	$640.00	$71.11	1.3%
Rent	$14,400.00	$1,600.00	29.1%

processed categories



Actionable tips based on number thresholds